Process Model Plug-ins

Preliminary

The contents of this document are preliminary and are subject to change without notice. This document may contain errors, omissions, or information that no longer applies.

To better understand the information in this document, National Instruments recommends that you familiarize yourself with the *Process Models* and the *Process Model Architecture* topics in the *NI TestStand Help*.

TestStand 2012 and later process models use sequence files called process model plug-ins to process test results and create result files. Each plug-in contains sequences TestStand calls during the process model execution to accomplish logging tasks. TestStand includes built-in plug-ins to handle report generation, database logging, and raw result logging. You can create custom process model plug-ins to extend the functionality of the TestStand process models. The plug-in architecture makes it easier to customize process models because you can install, modify, and remove process model functionality without changing the TestStand process model files directly.

Creating Process Model Plug-ins

Process model plug-in sequence files meet the following conditions:

- Located in the `<TestStand>\Components\Models\ModelPlugins` directory (built-in plug-ins) or the `<TestStand Public>\Components\Models\ModelPlugins` directory (custom plug-ins)
- Include a file global variable named `ModelPluginComponentDescription` of type NI_ModelPluginComponentDescription
- Sequence file type is **Model**
- Contain one or more plug-in entry point sequences.

TestStand uses the location of the sequence file and the existence of the `ModelPluginComponentDescription` file global variable to identify plug-ins. Because TestStand loads and inspects all the sequence files within the top level of the `ModelPlugins` directories, place any supporting sequence files that a plug-in calls in a different directory.

National Instruments recommends that you use a unique prefix that includes your company name, such as `Acme_ReportGenerator.seq`, for custom plug-ins you create. TestStand uses the root filename as a <*plug-in name*> prefix to plug-in data types it creates, such as `Acme_ReportGeneratorOptions`. National Instruments also recommends that you store any support files the plug-in requires in a subdirectory of the `<TestStand`

Public>\Components\Models\ModelPlugins directory with the same name as the plug-in file, such as ..\ModelPlugins\Acme_ReportGenerator\Acme_ReportGenerator_Support.seq.

Complete the following steps to generate a new plug-in that you can modify.

1. Select **Configure»Result Processing** to launch the Result Processing dialog box.
2. Enable the **Show More Options** control and click the **Advanced** button to launch the Advanced Result Processing Settings dialog box.
3. Click the **Create New Process Model Plug-in** button.
4. Use a unique filename for the plug-in and prefix the filename with your company name, such as Acme_ReportGenerator.seq. TestStand uses the root filename as a <*plug-in name*> prefix to plug-in data types it creates, such as Acme_ReportGeneratorOptions.
5. Click **Save** to create and load the new plug-in sequence file.

   TestStand creates and adds the following new data types to the file. The data types are empty containers in which you can add plug-in-specific properties.
   - <*plug-in name*>Options
   - <*plug-in name*>AdditionalOptions
   - <*plug-in name*>RuntimeVariables
   - <*plug-in name*>PerSocketRuntimeVariables

6. Close the open dialog boxes to view and edit the plug-in you just created.
7. Delete the entry point sequences the plug-in does not require. If you later determine that you need an entry point sequence you deleted, generate a new plug-in, copy the entry point sequence you need from the new plug-in, and add the entry point sequence to the plug-in that requires it.
8. Use an expression in the FileGlobals.ModelPluginComponentDescription.Default.Base.DisplayName subproperty to display a string that describes the plug-in or its output in the Output column of the Result Processing dialog box.
9. If you do not want plug-in end users to configure any plug-in-specific options, delete the Configure Standard Options and the Configure Additional Options sequences.
10. Complete the following steps if you want plug-in end users to configure plug-in-specific options.
    a. Add each option you want plug-in end users to configure as a subproperty of the <*plug-in name*>Options data type.
    b. Create a modal dialog box, in which plug-in end users can view and edit the plug-in options. Refer to the <TestStand Public>\Examples\ModalDialogs directory for examples of creating modal dialog boxes.
        i. Export a function, method, or VI to launch the dialog box and call the function, method, or VI in the Configure Standard Options sequence.
        ii. You can pass each <*plug-in name*>Options subproperty into and out of the dialog box entry point, or you can pass the plug-in or the <*plug-in name*>Options subproperty as a property object to use the TestStand

`PropertyObject` class methods to access the *<plug-in name>*Options subproperties in the dialog box code.

iii. Pass the value of the `RunState.Engine` property to the dialog box code so you can call the `Engine.NotifyStartOfModalDialogEx` and `Engine.NotifyEndOfModalDialog` methods to enforce the modality of the dialog box.

c. Optionally, you can use an expression in the `FileGlobals.ModelPluginComponentDescription.Default.Base.OptionsDescription` subproperty to display information in the Options column of the Result Processing dialog box. For example, the NI_ReportGenerator plug-in sets this expression to display the report format in the Options column of the Result Processing dialog box.

11. Consider whether you want the plug-in to always process information in a new thread, never process information in a new thread, or allow end users to specify whether to process information in a new thread. Typically, you allow processing in a new thread if the plug-in requires a significant amount of processing time and the plug-in can process the information while processing information from other instances of the plug-in and testing subsequent UUTs.

    According to your decision, set the following subproperties of `FileGlobals.ModelPluginComponentDescription.Default`:

    a. `Base.NewThread`
    b. `Base.CompleteBeforeNextUUT`
    c. `Base.UseDefaultNewThreadImplementation`

12. Save and close the plug-in when you have completed your customizations.
13. Complete the following steps to add an instance of the new plug-in to a result processing configuration. TestStand stores the configuration in `<TestStand Application Data>\ResultProcessing.cfg`. The process model loads this configuration file at run time to determine which plug-ins to invoke.
    a. Select **Configure»Result Processing** to launch the Result Processing dialog box.
    b. Click the **Add** button and select the plug-in name in the drop-down list.

When you insert a plug-in instance, the dialog box inserts a copy of the `FileGlobals.ModelPluginComponentDescription.Default` property values to use for that instance of the plug-in. Any changes you make to these values in the plug-in sequence file apply only to subsequent instances of the plug-in you create.

Refer to the Plug-in Data and Data Types section of this document for more information about plug-in properties.

Process Model Types

Plug-ins typically do not need to determine the type of process model that invokes the plug-in. However, plug-in entry points can inspect the `ModelType` subproperty of the **ModelData** parameter to determine the type of process model that invokes the plug-in. Possible values of the `ModelType` field for the built-in process models include `Sequential`, `Parallel`, or `Batch`.

For example, if the `ModelType` subproperty of the **ModelData** parameter is Batch, the built-in reporting plug-in synchronizes its controller and socket threads to ensure it generates the batch report from the controller thread before generating UUT reports from test socket threads so that reports appear in the expected order when the threads write the reports to the same file.

Process Model Thread Types

A process model can call plug-in entry points from multiple threads. The Batch model and the Parallel model create a separate thread and execution for each test socket and create a controller thread and execution to coordinate the activity of the socket threads. A process model calls some entry points from both socket and controller threads and other entry points only from socket threads or only from controller threads.

When a process model invokes a plug-in entry point, the model passes a parameter of type NI_ModelThreadType to the plug-in. The parameter includes two Boolean subproperties – `IsController` and `IsTestSocket`. Plug-ins can use these subproperties to determine what type of model thread invokes the plug-in. For example, a Begin entry point might perform a general operation when the controller thread invokes the entry point or a socket-specific operation when a socket thread invokes the entry point.

The Sequential model creates only one thread that coordinates and performs testing. The Sequential model sets the `IsController` and `IsTestSocket` subproperties to `True`.

Plug-in instance properties that change at run time are called run-time variables. A plug-in uses the following subproperties to store run-time variables specific to the plug-in:

- `<NI_ModelPlugin>.Base.RuntimeVariables` of type <*plug-in name*>RuntimeVariables
- `<NI_ModelPlugin>.Base.RuntimeVariables.PerSocket` of type array of <*plug-in name*>PerSocketRuntimeVariables

The process model sets the `PerSocket` property array to have one element for each test socket. Therefore, you can add a property to the <*plug-in name*>PerSocketRuntimeVariables data type to create a run-time variable that has a separate value for each socket thread.

When you use the Batch or Parallel process models to run a plug-in, the plug-in executes as a multithreaded sequence because the model invokes plug-in entry points in the controller thread and in socket threads. Therefore, you can access properties of the plug-in instance from multiple threads in parallel. TestStand ensures that reads and writes to TestStand properties

are atomic. However, National Instruments recommends that you use the following strategies to ensure that different threads safely share access to plug-in instance variables:

- Add run-time variables only to the <*plug-in name*>RuntimeVariables and <*plug-in name*>PerSocketRuntimeVariables data types.
- If a run-time variable must have a unique value for each test socket, add it to the <*plug-in name*>PerSocketRuntimeVariables data type.
- Use a TestStand lock or other synchronization method to ensure a thread has exclusive access to the plug-in run-time variables when required.

Step Results

A plug-in can receive step results after UUT completion or on-the-fly.

After UUT Completion

After each UUT completes, the process model calls the Model Plugin - Post UUT entry point and uses the **MainSequenceResult** parameter to pass a reference to the result of the process model Sequence Call step that calls the client file. The `Parameters.MainSequenceResult.TS.SequenceCall.ResultList` array contains the step results of the top-level client sequence. If a result in the array is for a Sequence Call step, the `TS.SequenceCall.ResultsList` property of the subsequence includes the results of all steps in the subsequence. The `TS.SequenceCall.Sequence`, `TS.SequenceCall.SequenceFile`, and `TS.SequenceCall.Status` properties include the sequence name, sequence file path, and result status for the subsequence.

A plug-in that processes results, such as a report generator or a database logger, typically recursively traverses the `TS.SequenceCall.ResultsList` array, including results of subsequences, to generate a report or log the data.

On-The-Fly

If you enable a plug-in to process results on-the-fly by setting the `ProcessOnTheFly` subproperty to `True`, TestStand collects results during test execution. When TestStand collects a result and exceeds a threshold in time or in the number of collected results, the process model passes the results collected since the last time TestStand exceeded the threshold to the Model Plugin - OnTheFly Step Results entry point for processing.

A plug-in that processes results on-the-fly, such as a report generator or a database logger, typically appends data to a report or logs the data to a database while iterating through each array of results the model passes to the OnTheFly Step Results entry point.

On-The-Fly - Provisional Results

TestStand cannot determine the result status of a Sequence Call step before the subsequence completes. Therefore, TestStand does not generate the final parent result for the steps of a subsequence until the subsequence completes.

However, to support plug-ins that must immediately identify the parent result for each child result, TestStand generates a provisional result for a Sequence Call step before executing the steps in the subsequence. The ID of a provisional result is the same as the ID of the corresponding final result. TestStand passes provisional results only to the Model Plugin - OnTheFly Step Results plug-in entry point and PostResults engine callback. Use the existence of the `TS.Provisional` Boolean subproperty to identify provisional results. In a Model Plugin – OnTheFly Step Results plug-in entry point, you use or ignore provisional results depending on what the plug-in requires.

Refer to the *Execution* topics in the *NI TestStand Help* for more information about TestStand results.

Structure of Plug-in Sequence Files

Plug-ins include specific entry points, entry point parameters, and data types.

Plug-in Entry Points

TestStand process models invoke plug-ins by calling a set of optional entry point sequences with a pre-defined set of parameters at specific points in the testing process. All entry point names begin with a `Model Plugin –` prefix. Implement only the entry points you need to perform specific tasks.

Model Plugin - Configure Standard Options

Use the Configure Standard Options entry point to provide a way to configure option settings for an instance of a plug-in. A plug-in that implements the Configure Standard Options entry point includes an icon in the Options column of the Result Processing dialog box. Plug-in end users click the icon to invoke the Configure Standard Options entry point, which launches a modal dialog box in which the end user can view and edit options for the instance of the plug-in.

Use the `Parameters.ModelPlugin.PluginSpecific.Options` subproperty to store the plug-in-specific options for the dialog box the entry point launches.

Note: If the plug-in implements the OnTheFly Step Results entry point and you want to provide a way to specify whether to process results on-the-fly, include a Boolean on-the-fly option in the dialog box and store the value of the option in the `ModelPlugin.Base.ProcessOnTheFly` Boolean property.

The Configure Standard Options entry point accepts the following parameters:

Parameter Name	Data Type
ProcessModel	Reference
ModelType	NI_ModelType
ModelPlugin	NI_ModelPlugin

Model Plugin - Configure Additional Options

Use the Configure Additional Options entry point to launch a second dialog box to provide a way for plug-in end users to configure option settings for the instance of a plug-in. Typically, you do not need this entry point because you can present options in a single dialog box, using tabs, modal popup dialog boxes, or other organizational methods to accommodate a large number of options.

Use this entry point when you create a new plug-in by extending an existing plug-in and you want to provide a small number of additional options for the new plug-in without changing the options dialog box for the original plug-in. The Configure Additional Options entry point launches a second dialog box that contains options specific to the new plug-in, and the Configure Standard Options entry point launches the unchanged options dialog box from the original plug-in.

If a plug-in implements the Configure Additional Options entry point, the Options column of the Result Processing dialog box includes a second settings icon immediately to the right of the icon that invokes the Configure Standard Options entry point.

Use the `Parameters.ModelPlugin.PluginSpecific.AdditionalOptions` subproperty to store the additional options for the enhanced plug-in.

The Configure Additional Options entry point accepts the same parameters as the Configure Standard Options entry point.

Model Plugin - Initialize

Use the Initialize entry point to perform run-time initialization of variables or options.

A process model calls the Initialize entry point once in its controller thread before calling any other run-time entry points and before calling the ModelPluginOptions and ModelPluginConfiguration callbacks.

Set the value of the `FileGlobals.ModelPluginComponentDescription.Default.Base.AlwaysInitialize` property to `True` for the process model to call the Initialize entry point and the ModelPluginOptions callback even if you do not enable the instance of the plug-in so that the plug-in and the client sequence file can dynamically enable or disable the instances of the plug-in.

If you do not set the value of the `AlwaysInitialize` property and the ModelPluginConfiguration callback enables a disabled instance of the plug-in, the process model calls the Initialize entry point and the ModelPluginOptions callback after the ModelPluginConfiguration callback returns.

The Initialize entry point accepts the following parameters:

Parameter Name	Data Type
ModelPlugin	NI_ModelPlugin
ModelPluginConfiguration	NI_ModelPluginConfiguration

Model Plugin – Begin

Use the Begin entry point to perform required setup or initialization tasks, such as opening files or establishing database connections.

A process model calls the Begin entry point once in its controller thread and once in each test socket thread and after calling the ModelPluginOptions and ModelPluginConfiguration callbacks, in contrast to the Initialize entry point, which the model calls only for the controller thread and before calling the ModelPluginOptions and ModelPluginConfiguration callbacks. Process models complete the call to the Begin entry point from the controller thread before calling the Begin entry point from any socket thread.

The Begin entry point accepts the following parameters:

Parameter Name	Data Type
ModelPlugin	NI_ ModelPlugin
ModelPluginConfiguration	NI_ModelPluginConfiguration
ModelThreadType	NI_ModelThreadType
ModelData	Container
StartDate	DateDetails
StartTime	TimeDetails
ProcessModelClientPath	Path
ParentThread	Reference

Model Plugin - Pre UUT

Use the Pre UUT entry point to perform required setup or initialization tasks before each UUT.

A process model calls the Pre UUT entry point once before each UUT in each test socket thread.

The Pre UUT entry point accepts the following parameters:

Parameter Name	Data Type
ModelPlugin	NI_ ModelPlugin
ModelPluginConfiguration	NI_ModelPluginConfiguration
ModelThreadType	NI_ModelThreadType
ModelData	Container
ProcessModelClientPath	Path
StartDate	DateDetails
StartTime	TimeDetails
UUT	UUT

Model Plugin – OnTheFly Step Results

Use the OnTheFly Step Results entry point to obtain step results during test execution. For example, when you enable the On-The-Fly Reporting option in the Report Options dialog box, the built-in reporting plug-in calls the OnTheFly Step Results entry point to append results information to the report during test execution.

A process model calls the OnTheFly Step Results entry point only for plug-ins configured to process results on-the-fly. The `<NI_ModelPlugin>.Base.ProcessOnTheFly` subproperty stores the on-the-fly setting. Teststand collects step results until the number of results exceeds a threshold value you specify and then passes the results through the process model to the OnTheFly Step Results entry point for processing. The number of results the entry point receives can vary depending on multiple factors, such as the options you set in the Advanced Result Settings dialog box.

A process model calls the OnTheFly Step Results entry point from any thread that generates results. Controller threads in the Batch and Parallel TestStand process models do not generate results.

Although a test socket thread can create multiple child threads, the TestStand process models prevent sockets from calling the OnTheFly Step Results entry point simultaneously for results from more than one thread in the same test socket. Therefore, the OnTheFly Step Results entry point can access any per-socket data without additional synchronization.

The OnTheFly Step Results entry point accepts the following parameters:

Parameter Name	**Data Type**
ModelPlugin	NI_ ModelPlugin
ModelData	Container
StartDate	DateDetails
StartTime	TimeDetails
UUT	UUT
Context	Reference
Steps	Reference Array
Results	Reference Array
CallbackNames	String Array
ParentIds	Numeric Array

The **Steps**, **Results**, **CallbackNames**, and **ParentIds** parameters are parallel arrays. For example, for the result element at `Results[5]`, the values of `Steps[5]`, `CallbackNames[5]`, and `ParentIds[5]` represent the step that generated the result, the callback name of the result, and the ID of the parent result for that result.

Model Plugin - Post UUT

Use the Post UUT entry point to process results after each UUT completes. For example, a plug-in might process the UUT test results and store the information in a report file or database tables.

A process model calls the Post UUT entry point once after each UUT for each test socket thread. The Batch process model ensures that the Pre UUT entry point completes in all test socket threads before invoking the Post UUT entry point in any test socket thread for the same batch.

Set the `<NI_ModelPlugin>.Base.NewThread` and `<NI_ModelPlugin>.UseDefaultNewThreadImplementation` properties to `True` for the process model to invoke the Post UUT entry point in a new thread.

The Post UUT entry point accepts the same parameters as the Pre UUT entry point and the following additional parameters:

Parameter Name	Data Type
UUTStatus	String
MainSequenceResult	Reference
CallingThread	Reference
NotificationToUnblockCallingThread	Reference
PostProcessingThreadForPriorUUT	Reference

Model Plugin - End

Use the End entry point to perform required cleanup or finalization tasks, such as closing files or database connections.

A process model calls the End entry point once in its controller thread and once in each test socket thread. Process models call the End entry point from the controller thread only after all calls to the End entry point from test socket threads complete. End is the last entry point the process model calls at run time after the UUT loop, if any, exits.

The End entry point accepts the same parameters as the Begin entry point.

Model Plugin – Pre Batch

Use the Pre Batch entry point to perform required setup or initialization tasks before each batch.

The Batch process model calls the Pre Batch entry point in the controller thread before each batch. The Batch model completes the call to the Pre Batch entry point from the controller thread before calling the Pre UUT entry point from any socket thread. The Parallel and Sequential process models do not call the Pre Batch entry point.

The Pre Batch entry point accepts the same parameters as the Pre UUT entry point.

Model Plugin - Post Batch

Use the Post Batch entry point to process results after each batch completes. For example, a plug-in might generate a batch report that summarizes the results of all test socket threads.

The Sequential and Parallel process models do not call the Post Batch entry point.

The Post Batch entry point accepts the same parameters as the Post UUT entry point.

Set the `<NI_ModelPlugin>.Base.NewThread` and `<NI_ModelPlugin>.UseDefaultNewThreadImplementation` properties to `True` for the process model to invoke the Post Batch entry point in a new thread.

Plug-in Entry Point Parameters

Plug-in entry points accept the following parameters.

Name	Data Type	Model Plug-in Entry Point	Description
CallbackNames	String Array	OnTheFly Step Results	Each element in this array specifies the callback sequence name of the step result at the corresponding index in the `Parameters.Results` array. Callback names are the same as subsequence result property names. The callback name is `SequenceCall` for steps that TestStand generates within a sequence you run or within the subsequences it calls using sequence calls.
CallingThread	Reference	Post UUT Post Batch	A reference to the socket or controller thread that invoked the Post UUT or Post Batch entry point. If the plug-in is configured to run in a new thread, the thread running the plug-in is a different thread from the thread the **CallingThread** parameter specifies.

Name	Data Type	Model Plug-in Entry Point	Description
Context	SequenceContext	OnTheFly Step Results	A reference to the `SequenceContext` for the model execution entry point that initiated testing. Because TestStand does not serialize the context to offline result files and does not pass the context to the OnTheFly Step Results entry point when processing an offline results file, do not use the **Context** parameter in a plug-in if an alternative implementation is possible. If a plug-in uses the **Context** parameter, set the `FileGlobals.ResultProcessor ComponentDescription. Default.CanProcessOffline OnTheFlyResults` property of the plug-in to `False` to indicate this situation.
MainSequenceResult	Reference	Post UUT Post Batch	A reference to the result of the process model call to the MainSequence callback in the client file. The results of the client file main sequence are located in the `MainSequenceResult.TS. SequenceCall.ResultList` property. TestStand does not currently use the value passed to the Post Batch entry point, and you can ignore it.
ModelData	Container The actual data type varies at run time depending on the process model. For built-in process models, the data type can be NI_SequentialModelData, NI_ParallelModelData2, or NI_BatchModelData2.	Begin End Pre UUT Post UUT Pre Batch Post Batch OnTheFly Step Results	Contains model-specific information, such as the `ModelOptions` property.
ModelPlugin	NI_ModelPlugin	All	Configuration and run-time variables for the plug-in instance.
ModelPluginConfiguration	NI_ModelPluginConfiguration	Begin End Pre UUT Post UUT Pre Batch Post Batch	Configuration and run-time variables for the set of all active plug-in instances. The **ModelPlugin** parameter is also an element within the `ModelPluginConfiguration. ModelPlugins` array property.
ModelThreadType	NI_ModelThreadType	Begin End Pre UUT Post UUT Pre Batch Post Batch	Specifies whether the current thread is a test socket thread, a model controller thread, or both.

Name	Data Type	Model Plug-in Entry Point	Description
ModelType	NI_ModelType	Configure Standard Options Configure Additional Options	String that indicates the type of the current process model. A dialog box that displays options that vary according to the type of model can use this value to determine which options to initially display.
NotificationToUnblockCallingThread	Reference	Post UUT Post Batch	Reference to a notification. If the value of this reference is a value other than `Nothing`, the entry point is executing in a different thread than the process model is executing in. The calling thread executing the process model does not proceed until the entry point sets this notification. This allows the entry point to copy parameter information that is subject to change if the process model proceeds to the next UUT or batch before the thread running the entry point completes. For example, the Pre UUT entry point in the built-in reporting plug-in creates and stores a new report in `ResultProcessor.RuntimeVariables.PerSocket[<socket index>].Specific.ReportInstance`. The Post UUT entry point in the built-in reporting plug-in copies the value of that property to a local variable before setting this notification.
ParentIds	Numeric Array	OnTheFly Step Results	Each element in this array specifies the result ID of the parent result for the result at the corresponding index in the `Parameters.Results` array. The parent is the result for the Sequence Call step that called the sequence that created the result. You can use parent IDs to reconstruct the hierarchy of sequence calls that generate results. The result for a process model MainSequence callback that invokes the `MainSequence` in the client file is at the root level of an execution and does not have a parent result. In this case, TestStand passes the negative value of the ID for the thread in which the step runs. TestStand negates the value so you can distinguish it from a result ID by the fact that is it less than zero.

Name	Data Type	Model Plug-in Entry Point	Description
ParentThread	Reference	Begin	When a test socket thread calls the Begin entry point, it passes its controller thread to the **ParentThread** parameter. If the calling thread is a controller thread, it passes `Nothing` to the **ParentThread** parameter even if the calling thread is also the test socket thread. Typically, a plug-in does not need to use the **ParentThread** parameter.
PostProcessingThreadForPriorUUT	Reference	Post UUT Post Batch	A reference to the thread that ran the entry point for the previous UUT or batch. The value is `Nothing` if the model did not invoke the previous call in a new thread. A plug-in can wait on this thread to ensure the model processes UUTs in chronological order. For example, if you configure the built-in reporting plug-in to store multiple UUT reports in the same file, the plug-in waits for this thread to complete before appending a new UUT report to the file in chronological order.
ProcessModelClientPath	Path	Begin Pre UUT Post UUT Pre Batch Post Batch	Absolute path to the client file the process model is executing.
Results	Array of References	OnTheFly Step Results	An array of references to the step result `PropertyObjects` TestStand has accumulated since the previous call to the OnTheFly Step Results entry point. Each element in the array is a TestStand reference. To access a result, you must dereference its reference.
StartDate	DateDetails	Begin Pre UUT Post UUT Pre Batch Post Batch OnTheFly Step Results	When passed to the Begin entry point, the **StartDate** parameter indicates when TestStand invoked the process model execution entry point. Otherwise, the **StartDate** parameter indicates when the UUT in the current socket began testing or when the current batch began testing for a batch controller thread.

Name	Data Type	Model Plug-in Entry Point	Description
StartTime	TimeDetails	Begin Pre UUT Post UUT Pre Batch Post Batch OnTheFly Step Results	When passed to the Begin entry point, the **StartTime** parameter indicates when TestStand invoked the process model execution entry point. Otherwise, the **StartTime** parameter indicates when the UUT in the current socket began testing or when the current batch began testing for a batch controller thread.
Step	Array of References	OnTheFly Step Results	An array of references to the steps that generated the corresponding elements of the `Parameters.Results` array. Each element in the array is a TestStand reference. To access a step, you must dereference its reference. Because TestStand does not serialize the steps to offline result files and does not pass the references to the OnTheFly Step Results entry point when processing an offline results file, do not use the **Step** parameter in a plug-in if an alternative implementation is possible. If a plug-in uses the **Step** parameter, set the `FileGlobals.ResultProcessorComponentDescription.Default.CanProcessOfflineOnTheFly Results` property of the plug-in to `False` to indicate this situation.
UUT	UUT	Pre UUT Post UUT Pre Batch Post Batch OnTheFly Step Results	For the Pre UUT, Post UUT, and OnTheFly Step Results entry points, the UUT parameter contains the unit under test information for the current test socket. For the Pre Batch and Post Batch entry points, the parameter contains unit under test information for the batch as a whole.
UUTStatus	String	Post UUT Post Batch	The result status of the UUT, such as `Passed` or `Failed`. TestStand does not currently use the value passed to the Post Batch entry point, and you can ignore it.

ModelPlugin Client-File Callbacks

A client file can override the ModelPluginOptions or the ModelPluginConfiguration model callback to inspect or alter the plug-in options an end user configures.

ModelPluginOptions

The model calls the ModelPluginOptions callback once for each plug-in instance and passes the instance to the **ModelPlugin** parameter. Use the subproperties of `Parameters.ModelPlugin.PluginSpecific.Options` to access plug-in-specific options. Use the `Parameters.ModelPlugin.Base.SequenceFileName` property to determine the plug-in to which the instance data belongs. The model calls the Model Plugin - Initialize entry point for each plug-in instance before calling the ModelPluginOptions callback so you can programmatically initialize or alter the plug-in instance before the model passes the instance to the client file.

The NI_DatabaseLogger and the NI_ReportGenerator plug-ins also provide plug-in-specific callbacks named DatabaseOptions and ReportOptions that client files can override to access plug-in-specific options.

ModelPluginConfiguration

The model calls the ModelPluginConfiguration callback one time after calling the ModelPluginOptions callback for every plug-in instance. The model passes the entire set of configured plug-ins to the **ModelPluginConfiguration** parameter. Iterate through the `Parameters.ModelPluginConfiguration.Plugins` array to access each configured plug-in.

Invoking Model Callbacks

Process models define model callbacks that client sequence files can override, such as the PreUUT or TestReport callbacks. Because a model plug-in is a model file, it can also define model callbacks that the client sequence can override.

The model plug-in file must include the model callback sequence so the plug-in can invoke the callback sequence. The process model file must also include a copy of the callback sequence for the sequence editor to display the callback sequence in the Sequence File Callbacks dialog box so you can conveniently override the callback sequence at edit time.

Plug-in Data and Data Types

Plug-ins use the following data types and subproperties.

NI_ModelPluginComponentDescription

Every plug-in file must include a `FileGlobals.ModelPluginComponentDescription` variable of type NI_ModelPluginComponentDescription. This variable includes the following fields:

- `Default` (NI_ModelPlugin) – The default structure and values for new instances of the plug-in. The process model copies this property to create a new instance of the plug-in in a configuration.
- `InsertOrder` (Number) – A value that determines where the plug-in appears in the list of plug-ins you can insert. The list sorts in ascending order. For example, the built-in reporting plug-in uses a value of `1`, and the built-in database plug-in uses a value of `2`.

Therefore, the built-in reporting plug-in appears before the built-in database plug-in in the list of plug-ins you can insert. You can specify fractional values.

- `ConfigurationOrder` - A value that determines where the plug-in appears in the default configuration of plug-ins the process model creates if the plug-in configuration file does not yet exist. The plug-ins appear in ascending order. For example, the built-in reporting plug-in uses a value of 1, and the built-in database plug-in uses a value of 2. Therefore, the built-in reporting plug-in appears before the built-in database plug-in in the default configuration. You can specify fractional values.
- `InitializationExpression` – The process model evaluates this expression when it instantiates a new instance of the plug-in. Use this expression to initialize fields that cannot use a static default value, such as system-dependent paths. Use the `ModelPlugin` prefix in the expression to refer to ModelPlugin fields. You can also access a top-level `UserCreated` Boolean property to determine if a user created the plug-in or if TestStand created the plug-in during the process of creating the initial configuration file. For example, the built-in offline results generator plug-in uses an expression similar to the following to specify the location of the default offline results directory in the `<TestStand Public>\Components` directory and to disable the offline results generator plug-in if you did not explicitly insert it.

  ```
  ModelPlugin.PluginSpecific.Options.Directory =
  Runstate.Engine.GetTestStandPath(TestStandPath_PublicComponents) +
  "\\Models\\UnprocessedResults",
  ModelPlugin.Base.Enabled = UserCreated"
  ```

- `DisplayNameExpression` – The process model evaluates this expression to determine the display name of the plug-in to use in the list of plug-ins you can insert. Use the `ResStr()` expression function to return a localizable string or specify a quoted string literal.

NI_ModelPlugin

When you insert an instance of a plug-in into a configuration, the process model creates the instance of the plug-in by copying the `FileGlobals.ModelPluginComponentDescription.Default` property of type NI_ModelPlugin in the plug-in sequence file.

When you edit the options for an instance of a plug-in, you change the values of subproperties of the NI_ModelPlugin instance.

Each time you execute a file using a process model, the process model loads a copy of the plug-in configuration from the configuration file. For each NI_ModelPlugin instance in the configuration, the model passes the instance to the **ModelPlugin** parameter of the entry points in the corresponding plug-in. The plug-in accesses the **ModelPlugin** parameter subproperties to determine the options you configured.

NI_ModelPlugin Properties

The NI_ModelPlugin data type includes `PluginSpecific`, `CategorySpecific`, and `Base` top-level subproperties.

PlugInSpecific

The `PlugInSpecific` property contains subproperties that vary depending on the plug-in. The `PlugInSpecific` property is an unstructured container, meaning the subproperties and their data types can vary regardless of the NI_ModelPlugin type definition.

When you create a plug-in, the `PluginSpecific` property includes the following subproperties, each with a data type unique to the plug-in. Edit the corresponding data type to add subproperties to each property.

- `Options` (<*plug-in name*>Options) – Create subproperties to store any options or settings the plug-in provides. Typically, you implement the Model Plugin - Configure Standard Options entry point to provide a way for end users to edit the options you add.
- `AdditionalOptions` (<*plug-in name*>AdditionalOptions) – Create subproperties only if you define a Model Plugin – Configure Additional Options entry point.
- `RuntimeVariables` (<*plug-in name*>RuntimeVariables) – Use the `RuntimeVariables` subproperty as the location to define variables the plug-in uses at run time. Create subproperties to share data within the same process model execution among the different entry points in the plug-in. The <*plug-in name*>`RuntimeVariables` data type must always include the following subproperty:
    - `PerSocket` (Array of <*plug-in name*>PerSocketRuntimeVariables) – At run time, the process model resizes the `PerSocket` array to include one element for each test socket in the system. By adding properties to the <*plug-in name*>PerSocketRuntimeVariables data type, you create per-socket variables you can use to share data within the same process model execution among the different entry points in the plug-in.

CategorySpecific

The `CategorySpecific` property provides a location for related plugs-ins to store information common to all plug-ins within a category. You do not need to edit `CategorySpecific` properties unless you are developing a set of related plug-ins that store common properties to support behavior that all plug-ins in the category share.

The `CategorySpecific` property includes the following subproperties:

- `Name` (String) – Specifies the category to which the plug-in belongs. Plug-ins with the same name belong to the same category. Code that implements category-specific functionality can inspect this field to determine the category of the plug-in.
- `Settings` (<*company prefix*>_<*category name*>CategorySettings) – The specific data type of category settings defines the option properties common to plug-ins in a category.

The built-in reporting, database, and offline results plug-ins set the `CategorySpecific.Name` property to `ResultProcessor` and set the data type of the `CategorySpecific.Settings` property to NI_ResultProcessorCategorySettings. You must use these settings so a plug-in appears in the Result Processing dialog box. TestStand uses these default settings for any plug-in you create from within the Result Processing dialog box.

Base

The `Base` property contains subproperties that specify functionality common to all plug-ins. The process model uses the subproperties of the `Base` property to determine how to invoke plug-in entry points and other aspects of plug-in operation.

The `Base` property includes the following subproperties:

- `Enabled` (Boolean) - Specifies whether the model invokes the instance of the plug-in at run time.
- `DisplayNameExpression` (Expression) - An expression that specifies the name to display in the Result Processing dialog box to identify the instance of the plug-in, such as `"<Company Name> Production Database"`. The value is an expression so that the default value the plug-in defines can refer to localized text using the `ResStr()` expression function. You can edit the text in the Output Name column of the Result Processing dialog box.
- `NewThread` (Boolean) - Specifies whether the plug-in performs lengthy operations in a separate thread.
- `UseDefaultNewThreadImplementation` (Boolean) - Specifies whether the process model automatically calls the Model Plugin - Post UUT and Model Plugin - Post Batch entry points in a new thread when the `NewThread` property is `True`. Set this property to `False` if the plug-in provides its own implementation for processing in a new thread when the `NewThread` property is `True`.
- `CompleteBeforeNextUUT` (Boolean) - Specifies whether the process model waits for the Post UUT thread of the plug-in to complete before calling any additional entry points for the current execution, except for the Post UUT entry points of other result processors. If the plug-in sets the `UseDefaultNewThreadImplementation` property to `False`, the plug-in implementation must honor the `CompleteBeforeNextUUT` setting.
- `ProcessOnTheFly` (Boolean) – Specifies whether the process model calls the Model Plugin - OnTheFly Step Results entry point.
- `CanProcessOfflineOnTheFlyResults` (Boolean) - Indicates whether the plug-in can process an offline results file written on-the-fly. Set this property to `False` for plug-ins that do not use the OnTheFly Step Results entry point or that access the **Step** or **Context** parameters within that entry point. The entry point must not access the **Step** or **Context** parameters because TestStand does not pass any data to these parameters when processing data from an offline results file.
- `CanProcessOfflineResultsTree` (Boolean) - Indicates whether the plug-in can process an offline results file that was not written on-the-fly. A value of `True` indicates that the Post UUT entry point completely processes results passed to it without requiring a call to the OnTheFly Step Results entry point.

- `OptionsDescriptionExpression` (Expression) - The result of the expression evaluation is the text you want to display to summarize the option settings in a configuration dialog box.

  Use the `ModelPlugin` prefix to access the properties of the plug-in instance in the expression. For example, you could use the following expression to display the path of a plug-in-specific output directory:

  ```
  "My plug-in stores its output files at: " +
  ModelPlugin.PluginSpecific.Options.MyPluginOutputPath
  ```

- `SequenceFileName` (String) – The process model sets this property to the filename of the sequence file that implements the instance of the plug-in.
- `IconName` (String) – Set this property to the name of an icon that represents the instance of the plug-in. The leftmost column of the Result Processing dialog box displays this icon. The icon name must be a valid argument to the `Engine.FindImage` method.
- `GUID` (String) – A string that uniquely identifies the instance of the plug-in.
- `AlwaysInitialize` (Boolean) – If this property is `True`, the process model calls the Model Plugin – Initialize entry point and the ModelPluginOptions callback for the plug-in even when the `Enabled` property is `False`. The entry point and callback can set the `Enabled` property to `True`. This option adds overhead for a disabled plug-in instance because the process model must still load the plug-in sequence file at run time.
- `RuntimeVariables` (NI_ModelPluginRuntimeVariables) – Variables the process model updates to control and indicate the execution state of the instance of the plug-in.

NI_ModelPluginRuntimeVariables

The NI_ModelPluginRuntimeVariables data type is the data type of the `Base.RuntimeVariables` property of the NI_ModelPlugin data type. The NI_ModelPluginRuntimeVariables data type contains variables the process model updates to control and indicate the execution state of the instance of the plug-in.

NI_ModelPluginRuntimeVariables Properties

The NI_ModelPluginRuntimeVariables data type includes the following subproperties:

- `ProcessorIndex` (Number) - Indicates the zero-based offset of the instance of the plug-in within the `ModelPlugins` array of the NI_ModelPluginConfiguration data type in which the instance resides. If you change the order of the `ModelPlugins` array, you must update the value of the `ProcessorIndex` property for the elements you move.
- `Thread` (Reference) - When the `NewThread` and `UseDefaultNewThreadImplementation` properties of the NI_ModelPlugin data type are `True` and the process model calls the Post Batch entry point in a new thread, the process model stores a reference to the new thread in this property if the parent thread is a controller thread.
- `PostBatchCompleteNotification` (Reference) – This property is reserved for future use.
- `PluginEntryPoints` (Number) – A bit mask that indicates which entry points the plug-in contains. The bit values correspond to the subproperties of the `FileGlobals.PluginEntryPoints` variable in `ModelSupport.seq`.

- `InitializationState` (Number) – A bit mask the process model uses to track whether certain run-time operations have occurred. The values correspond to the subproperties of the `FileGlobals.InitializationStates` variable in `ModelSupport.seq`.
- `PerSocket` (Array of NI_ModelPluginPerSocketRuntimeVariables) – At run time, the process model resizes the `PerSocket` array to include one element for each test socket in the system so that each socket has a separate instance of the NI_ModelPluginPerSocketRuntimeVariables data type.

NI_ModelPluginPerSocketRuntimeVariables

The NI_ModelPluginRuntimeVariables data type is the data type of the elements of the `Base.RuntimeVariables.PerSocket` array property of the NI_ModelPluginRuntimeVariables data type. The NI_ModelPluginPerSocketRuntimeVariables data type contains variables the process model updates to control and indicate the test socket execution state of the instance of the plug-in.

NI_ModelPluginRuntimeVariables Properties

The NI_ModelPluginPerSocketRuntimeVariables data type includes the following subproperties:

- `Thread` - When the `NewThread` and `UseDefaultNewThreadImplementation` properties of the NI_ModelPlugin data type are `True` and the process model calls the Post UUT entry point in a new thread, the process model stores a reference to the new thread in this property if the parent thread is a test socket thread.
- `PostUUTCompletionNotification` - This property is reserved for future use.

Enabling or Disabling Editing of NI_ModelPlugin Properties

Although, you can provide sub-dialog boxes for plug-in end users to edit plug-in-specific option properties, plug-in end users can also use the parent configuration dialog box to edit the following properties common to all plug-ins:

- `Base.Enabled`
- `Base.DisplayNameExpression`
- `Base.NewThread`
- `Base.CompleteBeforeNextUUT`
- `PluginSpecific.Options`
- `PluginSpecific.AdditionalOptions`

You can restrict or prevent plug-in end users from editing these properties by adding to a property an `NI.EnabledExpression` string attribute that contains a Boolean expression. For example, the expression `CurrentUserHasPrivilege("ConfigReport")` specifies that only users with the ConfigReport user privilege can edit the property.